

OST
Ostschweizer
Fachhochschule

Software Crafters

Das Warum und Wie von Dokumentationen

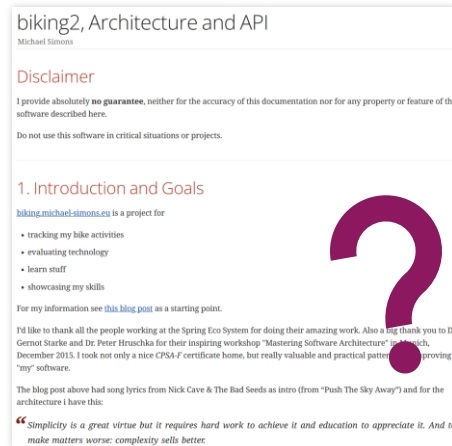
Lukas Messmer

2. Februar 2026

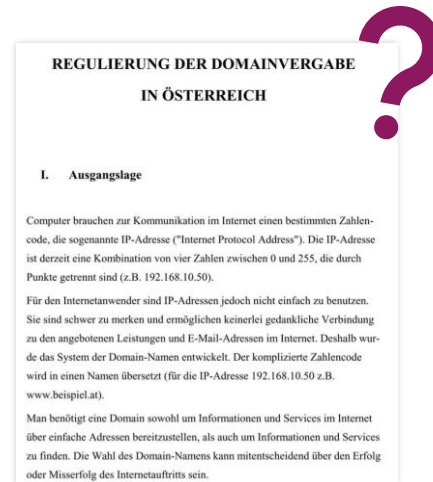
Einstieg

Warum dieses Thema?

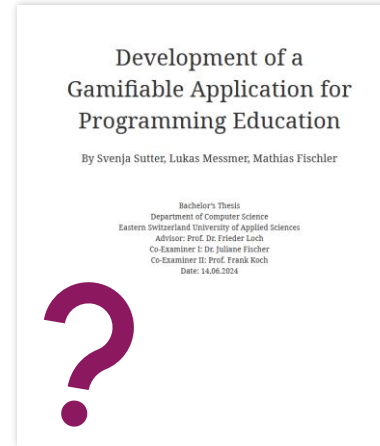
- Vorweg: Ich bin kein «Dokumentations-Guru»
- Ich habe jedoch verschiedene SA/BA-Arbeiten gelesen und weiterentwickelt.
- Dabei habe ich festgestellt:



Software
Dokumentation



Schulische
Dokumentation



SA/BA
Dokumentation

Was wollen wir erreichen?

- Aus meiner Sicht gilt ganz klar:

A software professional should be able to **write and maintain** documentation that is **consistent, accurate and easy to understand**.

- Viele «Altlasten» aus der Ausbildung hindern das ein wenig:
 - Dokumentation als Nebenprodukt (wenn überhaupt)
 - Schreiben um des Schreibens willen (z.B. Aufsätze)
 - Abhängigkeit von «Standardinhalten»
 - Kein tatsächliches Interesse an der Software (einmal abgegeben, ist es vorbei)

Wo fangen wir an?

- Wir wollen diese 3 Fragen beantworten:
 1. Was ist eine Software Dokumentation eigentlich?
 2. Welche Arten von Dokumentation gibt es?
 3. Wie können wir diese Arten verfassen?
- Die 3. Frage ist umfangreich und wir können sie mehrmals aufgreifen.
- Achtung: Dieser Talk ist «opinionated».

Was ist «Software Dokumentation»?



Was ist «Software Dokumentation»?

Definition

“Software documentation is written text or illustration that accompanies computer software or is embedded in the source code. The documentation either explains how the software operates or how to use it, and may mean different things to people in different roles.”

Wikipedia ^[1]

- Diese Definition enthält viele «or»...
- Software Dokumentation ist ein **Sammelbegriff**
- Es gibt zahlreiche unterschiedliche **Arten von Dokumentationen**

[1] https://en.wikipedia.org/wiki/Software_documentation

Was ist «Software Dokumentation»?

Welche Arten von Dokumentationen gibt es?

- Das ist gar nicht so eine einfache Frage...



Certainly! Here is a concise list of 10 types of software documentation:

1. **Software Requirements Specification (SRS)**
2. **Software Architecture Design**
3. **API Documentation**
4. **Technical/Source Code Documentation**
5. **User Manuals**

Documentation is an important part of software engineering. Types of documentation include:

- **Requirements** – Statements that identify attributes, capabilities, characteristics, or qualities of a system. This is the foundation for what will be or has been implemented.
- **Architecture/Design** – Overview of software. In construction principles to be used in design of
- **Technical** – Documentation of code, algorithms
- **End user** – Manuals for the end-user, system a
- **Marketing** – How to market the product and an

https://en.wikipedia.org/wiki/Software_documentation



<https://medium.com/@nomannayeem/mastering-the-art-of-software-documentation-a06aa5d7e697>

tion Guides

& Test Plans

Documentation

cumentation

Generiert von Google Gemini 3

Was ist «Software Dokumentation»?

Meine Übersicht

- Executives
- Project Lead
- Software Architect
- Quality & Compliance
- Developer
- End Users

High-Level For readers not directly involved in the software project (i.e. stakeholders)	Medium-Level For readers directly involved in the software project (i.e. architects)	Low-Level For readers developing, maintaining, or using the software (i.e. devs)
Management Summary ●	Project Documentation ●●●	API Documentation ●
Software Presentation ●	Requirements Specification ●●●	Technical Documentation ^[1] ●
	Architecture Documentation ●	DevOps Documentation ●
	Testing Documentation ●	Change- / Decision-Logs ●●
	Legal / Compliance ●	User Manuals ●●
	Security Policies / Audits / Reports ●	Training Materials ●●

[1] Includes things like code, databases, etc.

Was ist «Software Dokumentation»?

Wie unterscheiden sich Dokumentationen?

- Dokumentationen unterscheiden sich in 6 Aspekten:
 - **Zielgruppe:** Welche Personen interessieren sich für die Dokumentation?
 - **Format:** Wie soll die Dokumentation der Zielgruppe präsentiert werden?
 - **Autor:** Wer soll die Dokumentation schreiben und pflegen?
 - **Arbeitsmittel:** Wie soll die Dokumentation vom Autor verfasst werden?
 - **Inhalt:** Was soll die Dokumentation enthalten?
 - **Best Practices:** Was sollte man beim Verfassen dieser Dokumentation beachten?
- Diese Aspekte werden wir pro Dokumentationsart anschauen.

Was ist «Software Dokumentation»?

Welche Arten soll ich schreiben?

Don't try to use all documentation types available. **Different software solutions attract different kinds of documentation readers.** Only document what these readers actually want to know.

- Stellt euch immer Fragen zur Zielgruppe:
 - Gibt es externe Personen, die an meiner Software interessiert sind?
 - Sind noch andere Personen an meinem Projekt beteiligt?
 - Gibt es Personen, die meine Software nutzen werden?
 - Was werden diese Personen höchstwahrscheinlich tun?
 - Welche Informationen benötigen sie, um ... zu tun?
 - ...

Was ist «Software Dokumentation»?

Fallbeispiel «Open-Source Library»

Let's say I am creating a small open-source library that allows me to print Simpson quotes on the console, similar to Fortune. What kind of documentation does such a software need? Here is how you can determine this using a few simple questions and the framework provided above:

- *Are there external people who are interested in my software?*

Yes, there may be people who don't know this library and want to know what it is.

- *Are there people involved in my project?*

No, only myself.

- *Are there people who will use my software?*

Yes, there are.

- *What are these people most likely going to do?*

Probably use or extend my software.

- *What information do users need?*

Probably instructions on how to use the software.

- *What information do contributors need?*

Probably information about the code itself, e.g. where to find it.

High-Level	Medium-Level	Low-Level
Management Summary	Project Documentation	API Documentation
Software Presentation	Requirements Specification	Technical Documentation ^[1]
	Architecture Documentation	DevOps Documentation
	Testing Documentation	Change- / Decision-Logs
	Legal / Compliance	User Manuals
	Security Policies / Audits / Reports	Training Materials

Was ist «Software Dokumentation»?

The odd ones out (1/2)

- README
 - Oftmals der erste Kontaktpunkt zur Software
 - Sollte «ein wenig von allem» beinhalten
 - Enthält oft Referenzen auf andere Dokumentationen
 - Stellt ein funktionierendes «Minimal Working Example» bereit

<https://www.makeareadme.com>

Was ist «Software Dokumentation»?

The odd ones out (2/2)

- BA/SA-Dokumentationen
 - Oftmals eine Zusammensetzung aus unterschiedlichen Dokumentationsarten
 - Unterteilt die einzelnen Teile klar

High-Level	Medium-Level	Low-Level
Management Summary	Project Documentation	API Documentation
Software Presentation	Requirements Specification	Technical Documentation ^[1]
	Architecture Documentation	Issue Tracking
	Testing Documentation	Release Notes
	Legal / Compliance	User Manuals
	Security Policies / Audits / Reports	Training Materials



Inhaltsverzeichnis

1. Management Summary
2. Architecture Documentation
 1. ...
3. Project Documentation
 1. Assignment
 2. ...
4. Appendix
 1. Requirements Specification
 2. API Documentation
 3. Technical Documentation

Wie schreibe ich «Software Dokumentationen»?



Projektdokumentation

Eine Projektdokumentation ist eine Aufzeichnung von Zielen, Abläufen und Ergebnissen. Sie macht den gesamten Projektverlauf für alle Beteiligten und externe Personen transparent und nachvollziehbar.

- Alle Projektbeteiligten wissen, was sie wann und wie machen sollen / gemacht haben
- Kann unabhängig von der Software existieren
- Im besten Fall schreibt das jemand anderes 😊

Übersicht: Projektdokumentation

Zielgruppe Projektteam Stakeholder	Inhalt Projektauftrag / Vision Rollen / Verantwortlichkeiten Projektmethode / Kollaboration Zeitplan / Meilensteine / Budget Statusberichte Projektschlussbericht / Retrospektive
Format PDF-Dokument(e) Kollaborationssoftware	
Autor Projektleiter	Best Practices KISS (Keep it short and simple) Haltet dynamische Informationen in der Kollaborationssoftware Weitere Artefakte (z.B. Protokolle) separat halten
Arbeitsmittel WYSIWYG-Editoren (Word) Markup-Sprachen (Typst , Latex) Kollaborationssoftware (Jira, Confluence, Miro)	 Der Standard für Prozessmanagement: PMBOK Guide (PDF) Was man alles machen <i>könnte</i> : HERMES 2022 (Vorlagen)

Meine Arbeitsmittel

- Kleine Projekte: PDF-Dokument mit Typst
- Grosse Projekte: Kollaborationssoftware (Miro)

The diagram illustrates the structure of a project plan, organized into four main sections:

- Projektantrag & Vision:** Includes a slide titled 'Projektantrag & Vision' describing the project's goal (digitalizing a physics experiment) and a slide titled 'Forschungsfragen' (Research Questions) listing two specific questions about the experiment's feasibility and data processing.
- Organisation:** Includes a slide titled 'Organisation' showing a table of roles and responsibilities (e.g., Projektleitung, Projektschreiber, Projektsprecher) and a slide titled 'Risiken & Big Questions' showing a table of risks and big questions.
- Forschungsfragen:** Includes a slide titled 'Forschungsfragen' listing two specific research questions related to the experiment's feasibility and data processing.
- Projektplan:** Includes a slide titled 'Projektplanung' showing a Gantt chart with tasks like 'Projektmanagement' and 'Entwicklung des Experimentierkoffers' scheduled across quarters from Q1 2024 to Q4 2025.

Anforderungsspezifikation

Eine Anforderungsspezifikation definiert, was eine Software leisten muss, indem sie funktionale Anforderungen, Qualitätsmerkmale und Rahmenbedingungen festhält.

- Viele Online-Ressourcen verweisen auf Lasten- und Pflichtenhefte
- Ich habe noch nie eine sinnvolle «klassische» Anforderungsspezifikation gesehen...
- Darum: Keine Anforderungen in PDFs!



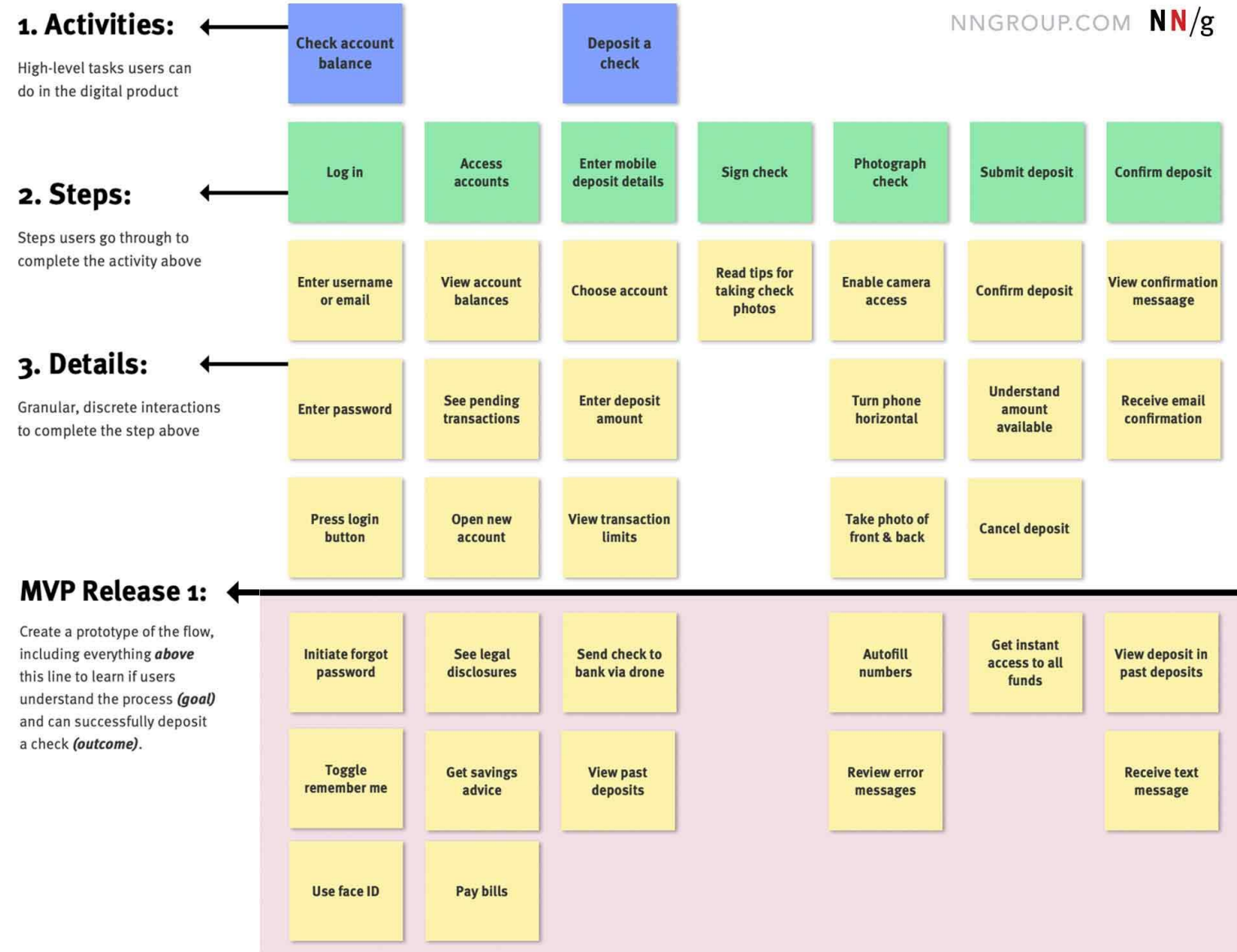
Übersicht: Anforderungsspezifikation

Zielgruppe Projektteam Stakeholder	Inhalt <u>User Stories / Epics / Acceptance Criteria</u> <u>User-Story Mapping</u>
Format Kollaborationssoftware	
Autor Projektteam	Best Practices Haltet eure Requirements dort, wo ihr auch arbeitet Achtet euch vor «Arbitrary Numbers» Verwendet «Agile Landing Zones»
Arbeitsmittel Kollaborationssoftware (Jira, Confluence, Miro)	

Requirements Specification

User-Story Mapping

- Mein neues Vorgehen für Anforderungsspezifikation
- Prozess:
 1. Unstrukturierte Erfassung
 2. Sortierung / Präzisierung
 3. Planung
- Vielversprechend



Was bei mir nicht funktionierte

- GitLab-Issues
 - Unübersichtlich
 - Verwaltung ist mühsam
 - Issues ≠ Tasks

Open 166 Closed 3 All 169

Search or filter results...

Plugins can have inputs (i.e. properties) provided by exercise makers.
ifs/user-centered-design/codingquiz/codingquiz_application#175 · created Apr 22, 2024 by Svenja Sutter Done FR MVP

The evaluation of an exercise can be triggered manually (multiple times).
ifs/user-centered-design/codingquiz/codingquiz_application#174 · created Apr 22, 2024 by Svenja Sutter FR MVP Optional Postponed

151 The application allows CRUD on courses.	
127 A new/empty course can be created.	Done
134 The files of a course can be imported from GitLab.	Done
135 An existing course can be imported from an archive/folder.	Postponed
152 An existing course can be edited.	Done
153 An existing course can be deleted.	
154 The administrative information of a course can be imported from Adunis.	Done
162 An existing course can be copied to create a new course.	
155 The application allows CRUD on weeks.	
156 A new/empty week can be added to a course.	Done
157 An existing week can be edited.	Done
158 An existing week can be deleted.	Done
159 An existing week can be restructured in its order.	

Architekturdokumentation

Eine Architekturdokumentation beschreibt die grundlegenden Strukturen, Komponenten und Designentscheidungen eines Systems. Sie erklärt den aktuellen Aufbau des Systems, macht komplexe Zusammenhänge verständlich und erleichtert künftige Erweiterungen sowie das Onboarding neuer Entwickler.

- Beinhaltet das, was viele unter «Software Dokumentation» verstehen.
- Aus meiner Sicht das wichtigste Dokument

Übersicht: Architekturdokumentation

Zielgruppe Software Architekten Entwickler Projektleiter	Inhalt Arc42 Template C4 Model Architectural Decision Records (ADRs) Y-Statements
Format PDF-Dokument Webseite	
Autor Software Architekten	Best Practices Dokumentiere nur «Architecturally Significant Decisions» Beschreibe keine Implementierungsdetails
Arbeitsmittel Markup-Sprachen (Typst , Latex) Versionskontrolle UML-Generatoren (Mermaid , PlantUML)	

Typst vs. Latex

- Philosophische Diskussion: Mir ist nicht wichtig, welches Tool ihr braucht
- Ich brauche Typst, weil ...
 - Einfache Syntax
 - Gute Dokumentation
 - Mächtiger als Markdown, AsciiDoc, etc.

<https://typst.app/docs/>

```
#set page(width: 10cm, height: auto)
#set heading(numbering: "1.")

= Fibonacci sequence
The Fibonacci sequence is defined through the
recurrence relation  $F_n = F_{n-1} + F_{n-2}$ .
It can also be expressed in _closed form_:

$ F_n = round(1 / sqrt(5) phi.alt^n), quad
phi.alt = (1 + sqrt(5)) / 2 $

#let count = 8
#let nums = range(1, count + 1)
#let fib(n) = (
  if n <= 2 { 1 }
  else { fib(n - 1) + fib(n - 2) }
)

The first #count numbers of the sequence are:

#align(center, table(
  columns: count,
  ..nums.map(n => $F_#n$),
  ..nums.map(n => str(fib(n))),
))
```

<https://github.com/typst/typst>

```
\documentclass{article}
\usepackage{amsmath}
\title{\LaTeX}

\begin{document}
  \maketitle
  \LaTeX{} is a document preparation system for
  the \TeX{} typesetting program. It offers
  programmable desktop publishing features and
  extensive facilities for automating most
  aspects of typesetting and desktop publishing,
  including numbering and cross-referencing,
  tables and figures, page layout,
  bibliographies, and much more. \LaTeX{} was
  originally written in 1984 by Leslie Lamport
  and has become the dominant method for using
  \TeX; few people write in plain \TeX{} anymore.
  The current version is \LaTeXe.

  % This is a comment, not shown in final output.
  % The following shows typesetting power of LaTeX:
  \begin{align}
    E_0 &= mc^2 \\
    E &= \frac{mc^2}{\sqrt{1-\frac{v^2}{c^2}}}
  \end{align}
\end{document}
```

<https://www.alba-translating.com/eng/eng/expertise/file-formats-eng/latex.html>

Was man nicht tun sollte

- Beschreibt keine trivialen oder irrelevanten Informationen

3.5.1 Datenbank Wahl

Für die Wahl der Datenbank kommt es in erster Linie darauf an, welche Datenbanken von Django unterstützt werden. Django unterstützt gemäss offizieller Dokumentation fünf Datenbanke

- PostgreSQL
- MariaDB
- MySQL
- Oracle
- SQLite

Zusätzlich ist es das Ziel, eine Datenbank zu verwenden, mit welcher in der Vergangenheit Erfahrungen gesammelt wurden. Dies ergibt folgende Schnittmenge mit Django: PostgreSQL, MySQL und SQLite. MariaDB und SQLite sind beide dateibasiert, sprich sie speichern die Datenbank innerhalb einer Datei. Dies vereinfacht die Handhabung während der Entwicklung. Da SQLite die Standarddatenbank von Django ist, ist SQLite ebenfalls in diesem Projekt verwendet.

SQLite
(Database)

SQLite was chosen for its simplicity and interchangeability. Using SQLite with Entity Framework (EF) Core allows for future drop-in replacement with other relational SQL databases (e.g. PostgreSQL, MariaDB/MySQL, etc.), which offer broader feature sets and have existing EF Core providers (Microsoft, 2024).
~~Additionally, unlike other databases, SQLite requires no setup or installation as it is simply file based.~~

In der folgenden Tabelle werden die verwendeten Typen erläutert.

Bezeichnungen	Erläuterung
BIGINT_PK	Ein Integer-Feld, welches von Django selbst angelegt und als primären Schlüssel verwendet wird.
NULL	Dieses Feld darf alternativ nicht ausgefüllt werden und wird dann mit dem None-Objekt befüllt.
BOOL	Ein Feld für einen binären Wert.
INT	Ein Ganzzahl-Feld mit normaler Grösse.

Architectural Decision und Y-Statements

- Konzentriert euch auf Informationen, die Einfluss auf eure Architektur haben
- Y-Statements von Olaf Zimmermann können dabei helfen
- Wenn man diese Y-Statements (ADR) durchnummeriert, erhält man ein Decision Log.

4.2.2. Automation & Dependencies

Y	In the context of the workload of exercise makers, facing the need to minimize it as much as possible (QA2), we decided to integrate as many external dependencies as possible (such as importing course information from Adunis), to achieve a minimal amount of manual effort for our functional requirements (Section II.A), accepting the downside of having additional dependencies that are outside of our control.
---	---

ADR 4: Why do we integrate with multiple external dependencies?

17.3. Architectural Decision Records (ADRs)

ADR 1: Why do we accept so many external dependencies?	8
ADR 2: Why are we implementing a plugin system?	10
ADR 3: Why is the plugin system limited to exercises?	10
ADR 4: Why do we integrate with multiple external dependencies?	11
ADR 5: Why do we use a Git server (Gitolite)?	11
ADR 6: Why do we implement a three-tiered architecture?	14

<https://adr.github.io/>

Was man nicht tun sollte

- Bildet keine technischen Details ab, insbesondere wenn sich diese häufig ändern
- Z.B.: Verwendet Domänenmodelle anstelle von Klassendiagrammen

